学习情境 L08：RAG 知识引擎与混合检索（6 学时）

一、学习情境基本信息

项目	内容
学习领域	LF3 智能查询服务开发：NL2SQL 与知识引擎
讲次	第 08 讲（第 8 周）
学时	6 学时（校内理实一体 4 学时 + 企业站实践 2 学时）
布鲁姆主层级	**应用 / 分析**（理解、创造渗透）
双元场地	校内：理实一体机房；企业站：DAP-v2 集群，Milvus `:19530` + CRDB 平台库
前置	L07 五步管线证据链完成；Milvus 两 collection（`dap_knowledge` / `dap_semantic_cache`）可写；嵌入服务可用
企业站 PR 任务	**PR-08**：为"售后域"建立知识索引（字段级 + 指标定义级 + 规则条目级三档 chunk，含 faq 元数据），跑 5 条验证问句 ≥4 条命中正确条目
课后费曼任务	学生讲堂③正式版：讲"为什么纯向量检索会翻车"，必须带一个亲手做出来的翻车案例

二、任务描述（客户委托单）

委托单编号：DAP-RAG-08

客服与业务侧反馈：洞察 Agent 常把"同义不同词"的问句答错，"订单号 O 开头怎么查"这类带编号的问句几乎必翻车。知识引擎组委托你们：走读三层混合检索（L1 精确 → L2 向量 → L3 关键词兜底）代码并画出决策流；完成向量+关键词互补实验与语义缓存 0.92 阈值权衡实验，用数据解释翻车根因；最后按字段级/指标定义级/规则条目级 chunk 粒度，为"售后域"建立可验证的知识索引。

三、学习目标

1. 【理解】解释三层检索的命中分工与兜底关系：L1 指标/表精确命中（含同义词归一）→ L2 Milvus 向量（cosine，faq 元数据透传）→ L3 CRDB `ILIKE` 关键词兜底（宁漏勿误）。
2. 【应用】走读 `retriever.py` / `semantic_cache.py`，画出含置信门槛与降级路径的检索决策流图。
3. 【分析】用对照实验证据解释"纯向量为何翻车"的两类根因：同义词漂移（知识库"销售额"vs 问句"营业额"）与编号/OOV 类查询（向量对编号不敏感）。
4. 【应用】完成语义缓存 0.92 阈值权衡实验：在 0.85/0.92/0.96 三档下测量命中率与错误命中率，解释 0.92 是"省 LLM 调用"与"答错题"之间的权衡点（LCO 铁律）。

5. 【创造】为售后域设计并落地三档粒度知识索引（含元数据 owner/updated_at/faq 标记），通过 5 条问句验证并复盘 chunk 粒度得失。

四、教学重点与难点

教学重点

1. 三层互补逻辑：每层的命中域、置信门槛、失败出口——以及"为什么必须有 L3"。
2. chunk 粒度策略：字段级（Schema 问答）/ 指标定义级（口径问答）/ 规则条目级（售后规则问答）各自适配的问句形态。

教学难点

1. 阈值权衡的本质：学生倾向认为"阈值越高越安全"。

突破策略：三档阈值实验让"命中率"与"错误命中率"两条曲线同屏——0.96 时几乎不命中（缓存白建），0.85 时把"退货率"当"退款额"答（错误命中），0.92 是本项目经验权衡点；再追问"什么业务该调高/调低"（答错代价高的调高，如口径类；闲聊类可低）。

2. chunk 粒度过粗/过细的检索损伤（过粗→单条向量语义稀释；过细→命中碎片答非所问）。

突破策略：同一售后规则文档分别按"整段"与"规则条目"两种切法灌入，用同一问句对比 top1 命中内容，亲眼看到整段切法返回的无关半句。

五、教学准备

类别	内容
环境	Milvus 可写（注意：`pymilvus` 插入主键必须 int64；`numpy>=1.26,<2.0`）；CRDB 平台库 `kb_chunks` / `kb_entities` 表就绪
代码	`services/insight-agent/app/rag/`：`retriever.py`、`semantic_cache.py`、`chunker.py`、`ingest.py`；`evals/sem_cache_sweep.py`；探针集 `evals/rag_probe_20.jsonl`（含 6 对"近义对"与 4 条"编号类"问句）
资料	售后域素材 `docs/knowledge/aftersale_rules.md`（6 条退换货规则）+ 售后指标口径草案；L07"未解决清单"汇总（本讲靶子）
工具	curl、jq、python CLI；决策流图模板（白板/在线绘图）
分组	沿用结对；每组领取一个"翻车复现任务"（同义词组 / 编号组二选一）

六、教学过程设计（六步法：资讯 30′/计划 30′/决策 20′/实施 90′/检查 40′/评价+学生讲堂 30′）

6.1 资讯（30′）

【教师活动】

1. 用 L07 留下的悬念开场，现场跑两条"必翻车"问句（纯向量开关打开）：

```
python -m services.insight_agent.rag.query --q "订单号 O 开头怎么查" --vector-only
# top1: "如何申请运费险理赔"（score 0.71）—— 编号 OOV，向量近似随机 → 翻车
python -m services.insight_agent.rag.query --q "营业额怎么统计" --vector-only
# top1: "利润口径说明"（score 0.78）—— 同义词漂移：营业额→利润 ≠ 销售额 → 翻车
```

2. 板书三层检索决策流（含门槛与出口）：

```
问句 q
 ├─ L1 精确命中：kb_entities（指标名/表名/别名 + 同义词归一表）
 │    "营业额"—同义词→"销售额"→ metric:gmv ✓ 直接返回（毫秒级、零模型调用）
 ├─ L2 向量：embed(q) → Milvus dap_knowledge（cosine top-k，faq 元数据随行透传）
 │    gate：top1.score ≥ 0.75 才采信，否则下沉          ← 置信下限
 └─ L3 关键词兜底：CRDB kb_chunks WHERE text ILIKE '%订单号%' LIMIT 5
      （宁漏勿误：命不中就明说"知识库未覆盖"，不硬答）
```

3. 讲解 chunk 粒度三档表（重点：粒度=检索单元=答案单元）：

粒度	切法	适配问句	风险
字段级	一字段一条（名称/类型/口径/示例值）	"pay_amount 是什么"	条目多、索引体积大
指标定义级	一指标一条（含计算式与排除项）	"GMV 口径含不含退款"	同义词必须维护进 L1
规则条目级	一条规则一条（如"上门取件费 8 元/单"）	"退货上门取件收费吗"	跨条目复合问题命中差

4. 点出语义缓存与检索的关系：`dap_semantic_cache` 是"问答对的向量缓存"，cosine≥0.92 才命中——它省的是整条 LLM 链路（LCO）。

【学生活动】 复画三层决策流；把自己组 L07 未解决清单里的失败问句标到三层图上（它会死在哪一层）。

【教学方法】 失败案例导入 + 板书决策流。

【设计意图】 用上一讲亲手制造的失败做锚点，"为什么需要三层"不再是设计者的说教而是学生自己的痛点。

6.2 计划（30′）

【教师活动】 指导排实验矩阵（三实验 + 一建设）：

序	实验	对照物	时间盒
R1	三层代码走读 + 决策流图	代码行号	20′
R2	纯向量 vs 混合对照（同义/编号两组问句）	vector-only 开关	25′

R3	语义缓存阈值扫描 0.85/0.92/0.96	命中率 vs 错误命中率双指标	20′
R4	售后域索引设计与试灌（校内先小样 10 条）	粒度三档	25′

【学生活动】 结对写计划：R2 要用的 5 条攻击问句（需覆盖同义词、编号、否定句三类）；R4 的 chunk 切分草案（在素材上直接划线）。

【教学方法】 实验矩阵 + 攻击用例先行（先设计"怎么考系统"再建设）。

【设计意图】 "攻击用例先行"是评测思维的雏形：学生先当出题人再当建设人，R4 的验证问句在建设前就定好，防止"建完再凑题"的自证陷阱。

6.3 决策（20′）

【教师活动】 审核两组关键决策：① R2 攻击问句是否真有区分度（否定句"退货不收运费对吗"会被 L3 命中吗？）；② R4 粒度选择是否与问句形态匹配。宣布写入纪律：灌索引走 `ingest.py`（幂等，带 `domain` 标签），**禁止**手工 insert 向量记录（int64 主键 + 元数据 schema 易错）。

【学生活动】 定稿攻击问句与切分草案；确认验证问句 5 条已写死进任务单（R4 不许改）。

【教学方法】 评审 + 冻结验证集。

【设计意图】 冻结验证问句 = 把"评价"环节前移到"决策"环节，这是 HSA（不可自证）的操作化。

6.4 实施（90′）

子步骤 1（20′）：R1 三层检索代码走读（主材料节选）

```
# services/insight-agent/app/rag/retriever.py（节选，走读版）
def retrieve(q: str, top_k: int = 5):
    # L1 精确命中：指标/表/别名 + 同义词归一（CRDB 表 kb_entities）
    if hit := exact_lookup(q):                          # "营业额"→归一→"销售额"→metric:gmv
        return [hit], layer="L1"
    # L2 向量：dap_knowledge，faq 元数据随行透传（source/owner/updated_at/kind）
    hits = milvus.search("dap_knowledge", embed(q), top_k,
                         output_fields=["text", "kind", "faq_meta"])
    if hits and hits[0].score >= 0.75:                  # 置信下限：低于则不采信
        return hits, layer="L2"
    # L3 关键词兜底：CRDB ILIKE，宁漏勿误
    rows = crdb.query("SELECT * FROM kb_chunks WHERE text ILIKE %s LIMIT 5",
                      f"%{keywords(q)}%")
    return rows, layer="L3"                             # 允许空：明说"知识库未覆盖"

# semantic_cache.py（节选）
res = milvus.search("dap_semantic_cache", embed(q), top_k=1)
if res and res[0].score >= THRESHOLD:                   # THRESHOLD=0.92
    return cached_answer(res[0]), hit=True              # 命中即跳过整条 LLM 链（~420ms→~40ms）
```

走读三问（任务单）：① L1 为什么必须带同义词归一表？（纯精确匹配救不了"营业额"）② 0.75 与 0.92 两个门槛为什么不同值？（一个是"采信检索结果"，一个是"复用答案"——后者错代价更高）③ faq 元数据透传给谁用？（生成层引用出处 + 运营侧定位过期知识）

子步骤 2（25′）：R2 纯向量 vs 混合对照实验

```
python -m evals.rag_compare --set evals/rag_probe_20.jsonl \
```

```
  --mode vector-only --out out/r2_vector.json
python -m evals.rag_compare --set evals/rag_probe_20.jsonl \
  --mode hybrid      --out out/r2_hybrid.json
jq '.summary' out/r2_vector.json out/r2_hybrid.json
# 示例（以实际为准）：
# vector-only: hit@1 = 0.55, 其中同义组 0.33 / 编号组 0.00
# hybrid:      hit@1 = 0.90, 同义组走 L1 归一全对 / 编号组走 L3 ILIKE 0.75
```

分析任务：把 4 类失败问句分别标注"死在哪一层、被哪一层救回"，填入对照表。

子步骤 3（20'）：R3 语义缓存阈值扫描

```
python -m evals.sem_cache_sweep --thresholds 0.85,0.92,0.96 \
  --set evals/rag_probe_20.jsonl --out out/r3_sweep.json
jq -r '.[] | [.threshold, .hit_rate, .wrong_hit_rate, .llm_calls_saved] | @tsv' out/r3_sweep.json
# 示例（以实际为准）：
# 0.85  hit=0.62  wrong=0.18   ← 把"退货率"当"退款额"答：错误命中不可接受
# 0.92  hit=0.48  wrong=0.02   ← 项目经验权衡点（LCO 铁律缺省值）
# 0.96  hit=0.15  wrong=0.00   ← 缓存几乎白建，LLM 成本照付
```

辩论题（任务单必答）：口径类业务（答错进财报）与闲聊类业务，分别该把阈值调向哪端？为什么？

子步骤 4（25'）：R4 售后域索引试灌（校内小样 10 条）

```
# ① 按三档粒度切分（chunker 支持字段/指标/规则三模式）：
python -m services.insight_agent.rag.chunker \
  --src docs/knowledge/aftersale_rules.md --granularity rule-item --out out/aftersale_chunks.jsonl
# ② 试灌 10 条（带 domain=aftersale 与 owner/updated_at 元数据）：
python -m services.insight_agent.rag.ingest \
  --domain aftersale --file out/aftersale_chunks.jsonl --limit 10
# ③ 用冻结的 5 条验证问句跑混合检索：
python -m services.insight_agent.rag.query --q "退货上门取件收费吗" --explain
#   layer=L2 score=0.83 text="上门取件服务费 8 元/单，买家承担…" faq_meta={owner:售后组,…}
# ④ 粒度对照：同一问句用"整段切法"重灌一次，对比 top1 是否漂移到无关半句
```

【教师活动】 巡回；在 R3 主持 5 分钟全班快辩（阈值两端派各 1 组发言）；收集 R4 的粒度对照结论（过粗切法 top1 漂移案例贴看板）。

【学生活动】 按矩阵执行；R2 失败标注表、R3 双指标表、R4 冻结问句成绩单三件证据入任务单；试灌阶段记录 int64 主键/元数据字段名等实操坑。

【教学方法】 对照实验 + 快辩 + 建设冻结验证。

【设计意图】 R2/R3 都以"两个指标同屏"呈现权衡（单指标必然诱导错误结论）；R4 冻结问句让"≥4/5 命中"成为不可作弊的验收线。

6.5 检查（40'）

【教师活动】 验收四件：① 决策流图（含两个门槛值与三出口）；② R2 失败标注表（每条失败问句归因到层）；③ R3 双指标表 + 辩论结论；④ R4 冻结问句成绩单（5 条逐条 top1 内容）。追问："L3 命中空结果时系统应该返回什么？"（答：明说未覆盖 + 引导提问模板，禁止硬答——宁漏勿误）。

【学生活动】 组间互查证据（重点：R4 是否偷偷改过验证问句——对照决策环节冻结记录）；登记缺陷与改进项转入企业站。

【教学方法】 冻结集核验 + 归因检查。

【设计意图】 "对照冻结记录查改动"是防自证作弊的标准动作，让学生体验评测的严肃性；未覆盖即明说，是知识引擎与"编答案"的分界线。

6.6 评价 + 学生讲堂（30’）

【教师活动】 按第八节评分；学生讲堂③："为什么纯向量检索会翻车"——随机抽 3 人，每人 3 分钟，必须带一个**亲手做出的**翻车案例（R2/R4 产物截图）与"哪一层救了它"；布置 L09 预习：观察 `dap_semantic_cache` 的命中指标与 FAQ 直答链路，思考"人工修正如何回流变成 few_shots"。

【学生活动】 带案例讲解；听众互评（案例真实/归因清楚/有数据 三项 ★1-3）；完成自评三问。

【教学方法】 费曼讲堂 + 证据化讲解（无案例不许讲）。

【设计意图】 "必须带亲手做的案例"把讲堂与实施强绑定，杜绝转述式演讲；预习直接衔接 L09 反馈闭环。

七、企业站实践任务卡（2 学时）

任务卡 PR-08：售后域知识索引全量建设与验证

项目	内容
任务目标	为售后域建成三档粒度知识索引（含元数据），冻结问句 5 条中 ≥4 条命中正确条目，并提交一条可复用的 FAQ 直答候选
前置条件	校内 R1-R4 完成、试灌 10 条无 schema 错误
素材	`aftersale_rules.md`（6 条规则全量）+ 售后指标口径 4 条（退货率/退款额/售后时效/一次解决率）+ 售后工单表字段清单（来自 L02 的售后域设计 PR）

步骤：

```
# 1. 三档全量切分与灌入（字段级 + 指标定义级 + 规则条目级，各自带 kind 元数据）
for g in field metric rule-item; do
  python -m services.insight_agent.rag.chunker  --src <对应素材> --granularity $g \
    --out out/aftersale_$g.jsonl
  python -m services.insight_agent.rag.ingest --domain aftersale \
    --file out/aftersale_$g.jsonl
done
# 2. 同义词进 L1：把"退货率/退货比例""退款额/退款金额"写入 kb_entities 别名表
psql "$DAP_CRDB_WRITE" -c "INSERT INTO kb_entities(name, kind, aliases, domain)
  VALUES ('退货率','metric','{退货比例,退单率}','aftersale') ON CONFLICT (name) DO NOTHING;"
# 3. 冻结问句验证（企业导师在场见证，逐条记录 layer/score/top1 摘要）
for q in "退货上门取件收费吗" "退货率怎么算" "一次解决率口径含不含二次工单" \
         "aftersale_ticket 表有哪些字段" "订单号 0 开头怎么查"; do
  python -m services.insight_agent.rag.query --q "$q" --explain | tee -a out/pr08_run.log
done
# 4. FAQ 直答候选：把验证中表现最好的一问一答写入 faq 候选表，供 L09 人工修正回流
```

交付物（PR 内容）： 索引构建脚本/命令记录、`out/aftersale_*.jsonl`（chunk 产物）、`pr08_run.log`（5 问句验证记录）、FAQ 候选 1 条、"未覆盖问句清单"（明说哪些问句知识库救不了）。

评审标准（企业导师三查）： 查粒度（三档齐且 kind 元数据正确）；查验证（5 问句逐条有 layer/score 证据，≥4/5 命中）；查诚实（未覆盖项如实列出，不许用低分 L2 结果冒充命中）。

双导师签栏	意见	签名/日期

校内导师		
企业导师		

八、评价设计（任务单评分表）

维度	权重	90-100 分标准	75-89 分标准	60-74 分标准	<60 分标准
决策流图（R1）	15%	三层+双门槛+三出口全对且标行号	缺一个门槛	缺出口	照抄板书
对照实验（R2）	25%	双指标对照 + 失败逐条归因到层	有对照无归因	单模式跑通	未做
阈值实验（R3）	20%	双指标表 + 辩论结论有业务分档理由	缺辩论结论	只有数字	未做
索引建设（R4/PR-08）	25%	三档齐 + 冻结问句 ⩾4/5 + 未覆盖如实列出	3/5	2/5 且有凑题嫌疑	<2/5 或改题
学生讲堂（评价）	15%	亲手案例 + 归因 + 救援层讲清	案例非亲手	转述概念	讲不出

红线：验证阶段修改冻结问句、或把 L2 低分结果记为"命中"，本讲 0 分重做（HSA/评测纪律）。

九、课后费曼任务

项目	内容
讲给谁	同宿舍/同班不选这门课的同学（懂一点编程但不懂 NLP 即可）
讲什么	6 分钟讲"为什么纯向量检索会翻车"：① 带你亲手做的翻车案例（截图）；② 讲清两类根因（同义词漂移、编号 OOV）；③ 讲"哪一层救了它"（L1 同义词归一 / L3 关键词兜底）；④ 附加题：用"查字典"类比解释为什么 chunk 切太粗会答非所问
自评三问	① 我的翻车案例数据（score/layer）拿得出来吗，还是凭印象说的？② 我能不能说清 0.75 和 0.92 两个门槛为什么不相等？③ 如果老板说"把阈值调到 0.99 保证不出错"，我能用 R3 的双指标数据反驳他吗？

十、教学反思栏

反思维度	记录（课后填写）
翻车案例的区分度（攻击问句质量）	

阈值辩论中业务分档意识的达成	
粒度过粗切法的 top1 漂移案例汇总	
PR-08 冻结问句成绩分布（x/5）	
未覆盖问句清单（转 L09 扩题素材）	